

本サンプルは、旧情報工学科の実験科目で学生が1人で（チームではなく）実施したプロジェクトの実際の報告書をハッカソン用の報告書に見えるように追記・修正したものである。データや考察等はオリジナルのままであるが、スケジュール等は架空のものである。あくまでサンプルであり、これが十分なクオリティとは認識しないこと。

Arduino と光センサを用いた 居眠り検知・防止装置

情報工学科 2 年

24G1000

工大 太郎

—— チーム 116 ——

工大 二郎 (24G1001) 工大 三郎 (24G1002)

工大 四郎 (24G1003) 工大 五郎 (24G1004)

2025 年 6 月 24 日

1 はじめに

日中に眠気を感じる人は多い。厚生労働省の令和元年国民・栄養調査報告 [1] によると、男性の 32.3%、女性の 36.9%が日中に眠気を感じていることが報告されている。適度な仮眠は集中力を高めたり、脳を活性化させることが知られているが、業務の形態や進捗によっては仮眠が許されないことも多いし、意図しない居眠りが業務の遅れや事故に繋がることもある。特に昼食後のデスクワークにおける居眠りを防ぐためには、システム面から居眠りの検知・防止ができることが望ましい。

これまで、居眠り（覚醒度低下）検知システムの開発は、運転中における居眠りなどを対象として数多く行われてきた。既存の方法は、体動の変化から居眠りを検知する方法と生理指標から居眠りを検知する方法の大きく二つに分けられる。前者には、画像処理による人の瞬目の変化からの居眠り検知 [2]、加速度センサによる頭の動きからの居眠り検知 [3]、座席の背もたれに設置した導波管型センサからの居眠り検知 [4] などがある。後者には、心拍の時系列解析による居眠り検知 [5]、脳波からの居眠り検知 [6] などがある。いずれも眠気と相関関係のある物理量を計測し、居眠りを推測している。特に脈拍センサによる居眠りの検知方法は、ストレス値を定量的に評価したり、視覚化したりできるため、先行研究も多い [5][7][9]。ところで、このセンサは、指に正しく装着されていない場合や意図せず指から外れていた場合に乱数値が出力されることがある。しかし、そのような場合にそれが使用者の誤った使用方法に起因していることをセンサ側から検知できないという技術的課題がある。

そこで本プロジェクトでは、体動の変化からの居眠り検知の新たな方法として、光センサを用いた居眠り検知・防止を提案する。これは、昼食後のデスクワークにおける居眠り検知・防止を想定している。具体的には、頭の動きを光センサで捉え、頭の影と環境光との光量の差から居眠りを検知する。居眠りを検知すると、モータによって振動を起こし、使用者を起床させる。また、居眠り検知の既存の方法の技術的課題の解決として、使用法に誤りがあることを検出し、使用者に警告する機能を加える。

また、本プロジェクトでは、脈拍センサを用いて使用者のストレス状態を視覚化した上で、光センサによる居眠り検知が正しく行われているかを検証した。具体的には、睡眠状態の解析に用いられるローレンツプロット (LP: Lorenz Plot) [7][8] を Processing によって表示し、使用者のストレス状態を幾何学的に視覚化した。さらに、松本佳昭らの論文で提案されている評価指標 F-stress[9]、心拍間隔 (RRI: R-R Interval)、二つの光センサの変動について使用者の状態ごとに記録し、比較して解析した。LP 情報と実験の結果より、本装置によって使用者の居眠り検知・防止ができることを明らかにする。さらに、本装置は F-stress では取得できない使用者のうとうと状態をグラフ上に波形として捉えられることが可能であることも明らかにする。本システムは、この点において既存の評価指標よりも有効性があると言える。また、誤使用時や暗所時においても想定通りの動作が正しく行われていることを明らかにする。

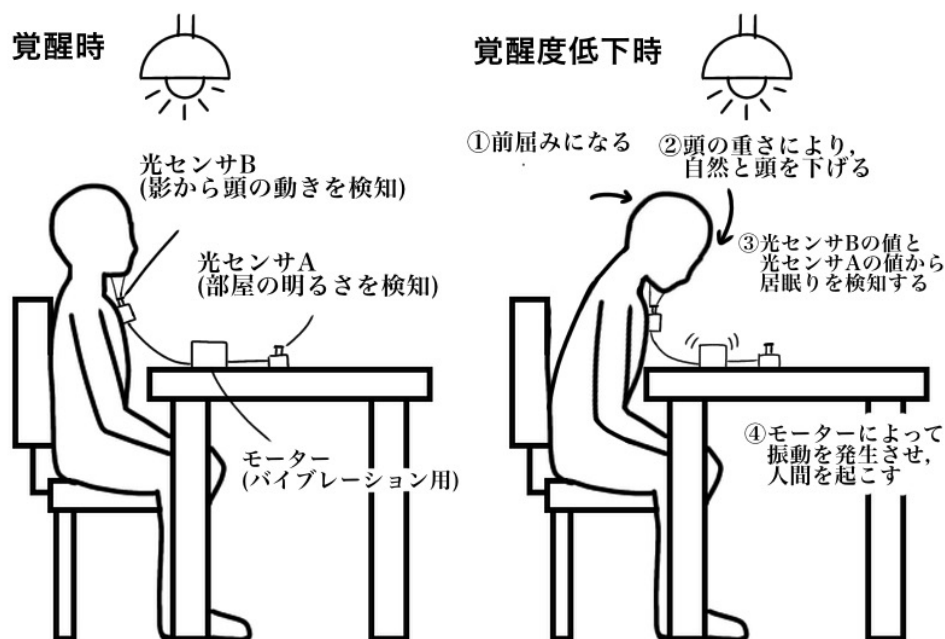


図1 覚醒時と覚醒度低下時のシステムの動作

2 作成したシステムの概要

2.1 光センサによる居眠り検知

2.1.1 システムの概要

光センサによる居眠り検知・防止装置の大まかな仕組みは、図1に示す通りである。この装置は、2つの光センサ（Cdsセル）を使用する。これまで覚醒度低下の検出に光センサを用いた例は見当たらず、簡易に実現できるため、この点において本装置は新規性があると考えられる。光センサの一つ目は、部屋の明るさを検知するための光センサAである。光センサAは、図1のように物体の影にならないように配置する。しかし、光センサAの値は、他人が傍を通るだけで容易に変化してしまう。よって、システムの各条件に用いる環境光（Ambient_Light）は、光センサAの分散が一定以上小さくなった時の光センサAの平均値とする。予備実験として、本装置を用いて動作確認をした結果、光センサAの値が安定した時の分散の値は、0または1であったため、光センサAの分散の条件は1以下とした。二つ目は、頭の動きを検知する光センサBである。光センサBが設置されたボックスには安全ピンがついており、衣服に装着して使用する。

まず、覚醒度低下時のシステムの動作を説明する。図1の右側のように、覚醒度低下時は、人は

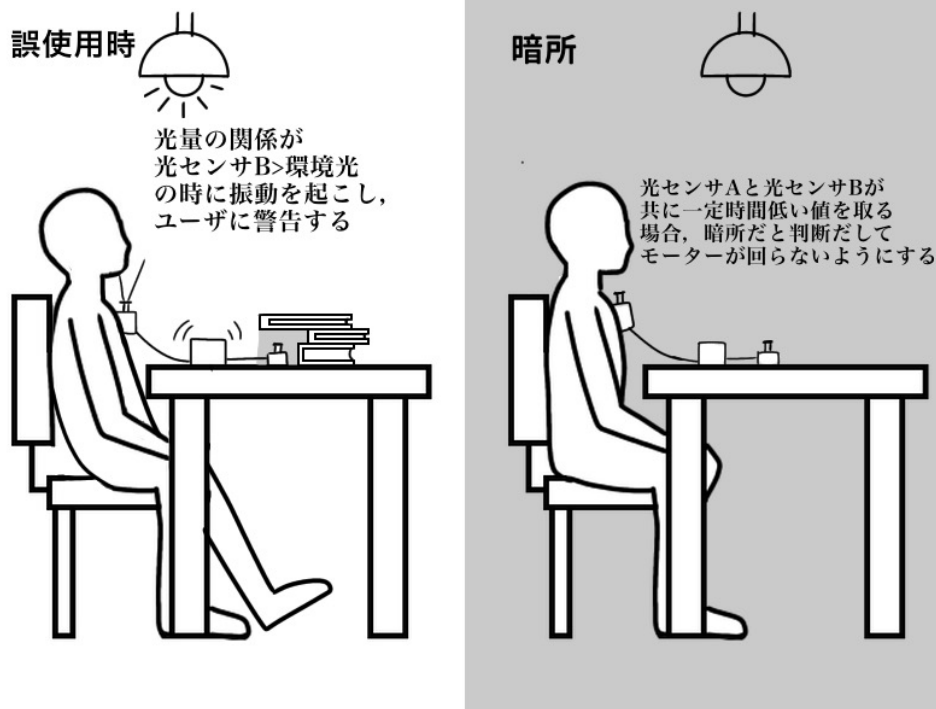


図2 姿勢が悪い時と暗所にいる際のシステムの動作

頭の重さによって下を向いたり前かがみになることが自然である。前かがみになると、光センサ B の値が環境光の光センサ A の値よりも小さくなる。本システムはこの値の差を用い、2.1.2 節で説明する方法によって居眠り状態を検出する。居眠り状態が検出されると、モータを回転させて振動を起こし、使用者の居眠り防止を図る。

次に、誤使用時のシステムの動作を説明する。ここでいう“誤使用時”とは、図2のように使用者の姿勢が悪い時、光センサ A が物影にある時などを指す。いずれも光センサ B の値が環境光よりも大きくなるような場合である。この時、光センサによる居眠り検知・防止は正しく行えないため、モータによる振動で使用者に問題を解決するよう警告する。この点において、誤使用を検知できずに乱数値を出力する脈拍センサよりも有効性があると言える。また、正しく装着していても頭を動かすことにより、光センサ B が環境光よりも値を大きく取ってしまう場合がある。そのような状況と誤使用を区別するため、条件として5秒間毎の光センサ B の分散（頭の動きの大きさ）が1以下であることを加える。

最後に暗所にいる際のシステムの動作を説明する。本装置は、昼食後のデスクワークにおける居眠り検知・防止を想定しているため、暗所時は動作しないこととする。具体的には、5秒間の光センサ A の平均値と光センサ B の平均値がともに閾値（Night_Mode_Value）より小さい場合に暗所と判断し、誤ってモータが回らないようにする。実際には、予備実験により暗いと感じた時の光センサの値を調べ Night_Mode_Value は 15 と設定した。

表 1 環境光と頭の影の光量の関係

環境光の平均値	光センサ B の値の平均値	環境光の平均値/光センサ B の値の平均値
54.2	19.8	2.73
42.6	14.4	2.94
32.0	10.4	3.07
24.2	7.8	3.1

表 2 環境光と下を向いた頭の影の光量の関係

環境光の平均値	光センサ B の値の平均値	環境光の平均値/光センサ B の値の平均値
55.6	13.2	4.21
43.8	9.4	4.65
33.0	8.4	3.928
25.0	7.6	3.28

2.1.2 頭の影と環境光の関係と居眠り判定法

光センサによる居眠り判定の条件を考案するために、頭の影と環境光の光量の関係を導いた。二つの光センサ（Cds セル）を用意し、2.1.1 節で説明したシステムと同様に机の上に光センサ A を、首元に光センサ B を設置した。正しい姿勢を維持したまま、部屋の明るさ（環境光）のみを 4 段階変化させ、二つの光センサの光量を 5 分おきに 5 回記録した。それぞれの段階の平均値の結果を表 1 に示す。表 1 に示された“環境光の平均値/光センサ B の値の平均値”の 4 つの結果をみると、環境光と光センサ B の値は一定の比率があることが読み取れる。“環境光の平均値/光センサ B の値の平均値”をさらに平均すると 2.96 となる。ここから、実験を行った部屋における環境光の光量は、頭の影の光量の 3 倍であるという関係性があることがわかる。また、部屋の明るさによらず、環境光と頭の影の比率は、変わらないことも読み取れる。この結果から、光センサによる居眠り検知の判定方法は、光センサ B の値が環境光に比べ、何倍であるかに注目すればよいことがわかる。そこで、居眠りの体勢をとった状態における光センサ B の値と環境光の値を同様の条件で計測した。その結果を平均したものを表 2 に示す。

表 2 に示された“環境光の平均値/光センサ B の値の平均値”の 4 つ結果をさらに平均すると 4.017 となる。ここから、実験を行った部屋における環境光の光量は、頭の影の光量の 4 倍であるという関係性があることがわかる。よって、居眠り判定の閾値を環境光の 4 分の 1 とした。また、光センサ B の値と環境光の比較のみを居眠りの条件としてしまうと、覚醒時に頭を動かした際に誤って居眠りと判定してしまう場合がある。使用者がうとうとして前かがみになり、頭が動かなくなった時点居眠りと判定するため、居眠りの判定条件に 5 秒間ごとの光センサ B の値の分散が 1 以下であることを加えることとする。

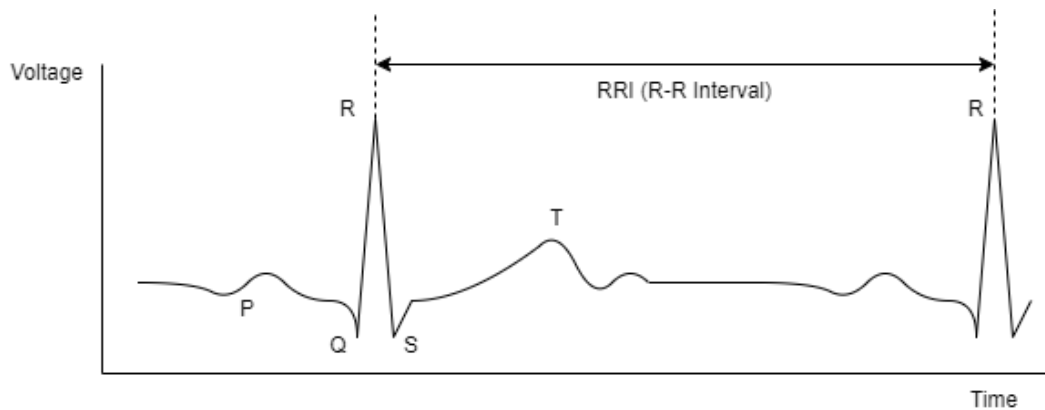


図3 RRI (R-R Interval)

2.2 脈拍センサによる居眠り推定法

2.2.1 心拍間隔の定義

居眠り検知に使われている生理情報の一つとして、心拍変動（HRV: heart rate variability）がある。HRV は、心電図や脈波等から計測した心拍周期の変動を解析するものである。心電図（ECG: Electrocardiogram）の概略図を図3に示す [7][9]。この電気信号は、心臓が収縮する際に生じる電気の分布の変化によって引き起こされる [10]。この電気信号には、P、Q、R、S、Tといった名称がつけられており、特に図3に示す波形のピークはRと呼ばれる。一般にR波は、心拍の間隔を計測する際に利用される。R波同士の時間間隔のことをRRI（R-R Interval）と呼び、これが心拍間隔に相当する。

2.2.2 LPとは

HRV解析の時間領域指標の一つにローレンツプロット（LP: Lorenz plot）がある。LPは、ポアンカレプロットとも呼ばれる。これは、RRIの変動を視覚的に捉える幾何学図形解析手法であり、不整脈自動解析、心臓自律神経機能解析、睡眠状態の解析等に用いられている [9]。具体的には、図4に示すように、 n 番目のRRIをX座標、 $n+1$ 番目のRRIをY座標とし、心拍に対してx-y平面上に順にプロットしたものである。通常、LPは直線 $RRI_{n+1} = RRI_n$ を長軸とした楕円状に分布することが知られている。また、一般に安静時はプロットの重心が右上に推移し、緊張時は左下方向に推移しながら各点の散らばりが大きくなる。さらに緊張が高まると、プロットの重心は左下に推移しながら各点の散らばりが小さくなることが知られている。

2.2.3 F-stressの実装

2.2.2で説明したLPの性質を考慮し、松本佳昭らは評価関数F-stressを提案した [9]。まず、(1)式に示すように、プロットの原点から重心までの距離を g_p とする。さらに、(2)式で示すように、重心と各点のばらつきを表す指標として、各点から重心までの平均距離を δ とする。これらの特徴

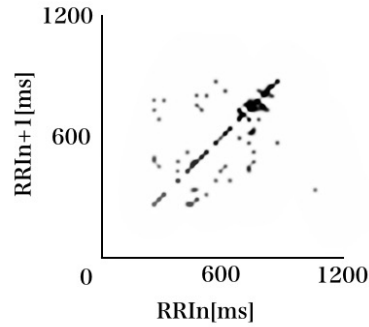


図 4 ローレンツプロット (LP)

量を (3) 式のように積をとったものを F-stress とする.

$$g_p = \sqrt{X_G^2 + Y_G^2} \quad (1)$$

ただし,

$$X_G = \frac{1}{n} \sum_{i=0}^{n-1} X_i, Y_G = \frac{1}{n} \sum_{i=0}^{n-1} Y_i \quad (2)$$

$$\delta = \frac{1}{n} \sum_{i=0}^{n-1} \sqrt{(X_G - X_i)^2 + (Y_G - Y_i)^2} \quad (3)$$

$$F_{\text{stress}} = g_p \cdot \delta \quad (4)$$

ここで, 論文 [9] では g_p と δ は, それぞれ 0~1 に正規化したものを用いている. しかし, 本実験では, Processing で F-stress の変動をより見やすくするために 0~1 正規化を行わないこととする. また, Processing での表示の都合上, Arduino 側で F-stress を 5 で割った値を Processing に送信するようにしている. これは, 表示するためだけに値の大きさを変更しているため, 本来の値の変動自体に影響を与えない. 論文 [9] によると, この評価値がゼロに近くなるほど, ストレスが高まっていることになる. Arduino 側での F-stress を算出する関数を図 5 に示す.

2.3 システムの実装

2.3.1 フローチャートと関数の機能

Arduino 側の主要な処理で用いる関数を次のように定義する.

loop() 関数 センサから値を読み取り, その値を送信できる形に変換する. その後, シリアル通信を用いて Processing へ送信する. Check_F_stress() や Check_Data() などの各関数を呼び出す. 尚, この関数は繰り返し実行される. フローチャートを図 6 に示す.

```

1:   int Check_F_stress(){//ストレス値を計算
2:       float delta=0;
3:       float G_p,G_x,G_y;//G_pは重心
4:       int F_stress;
5:       int sum_x =0,sum_y=0;
6:       int i;
7:
8:       for(i = check_range - 2; i >= 0 ;i--){
9:           sum_x += Data_IBI[i];
10:          sum_y += Data_IBI[i+1];
11:      }
12:      G_x = sum_x/(check_range - 1);
13:      G_y = sum_y/(check_range - 1);
14:      G_p = G_x*G_x + G_y*G_y;
15:      G_p = sqrt(G_p);
16:      for(i=check_range - 2; i >= 0 ;i--){
17:          delta = delta + (G_x - Data_IBI[i])*(G_x - Data_IBI[i])
18:              + (G_y - Data_IBI[i+1])*(G_y - Data_IBI[i+1]);
19:      }
20:      delta = sqrt(delta);
21:      delta = delta /(check_range - 1 );
22:      F_stress = delta*G_p/5;
23:      return F_stress;

```

図 5 スケッチ 1: F-stress の関数

Check_F_stress() 関数 RRI から計算できるストレス値を算出する。フローチャートを図 7 に示す。

Check_DataA_Variation() 関数 5 秒間の光センサ A の平均値と 5 秒間ごとの分散を求め、分散が 1 以下になった時の平均値を環境光として設定する。フローチャートを図 8 に示す。

Check_DataB() 関数 5 秒間の光センサ B の平均値と 5 秒間ごとの分散を求め、暗所モード判定、居眠り判定、誤使用判定等を行う。フローチャートを図 9 に示す。

2.3.2 Processing 側での表示の工夫

多くのデータをシリアル通信していると値にノイズが入ることがある。例えば、光センサ B の値の分散 (Data_B_var) を Processing 側で表示すると、“-1” という値が画面上でチラつくことがある。そこで、図 10 に示すように工夫を施した。Data_B_var が -1 であるならば、過去の B の値の分散 (Data_B_var_pr) を表示し、Data_B_var が -1 でないならば、Data_B_var_pr を更新して現在の B の値の分散 (Data_B_var) を表示する。こうすることにより、-1 のチラつきを抑えることができた。

2.3.3 回路図と用いた機器

用いた機器を表 3 に示す。また、回路図を図 11 に示す。

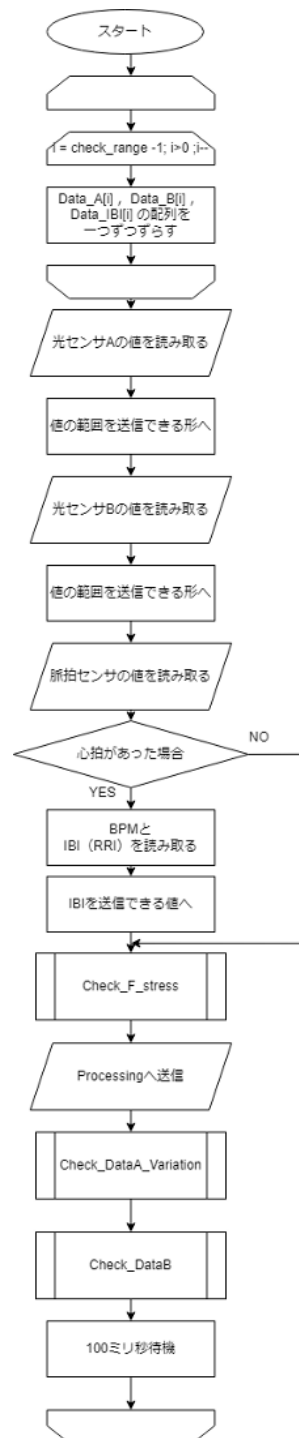


図 6 loop() 関数のフローチャート

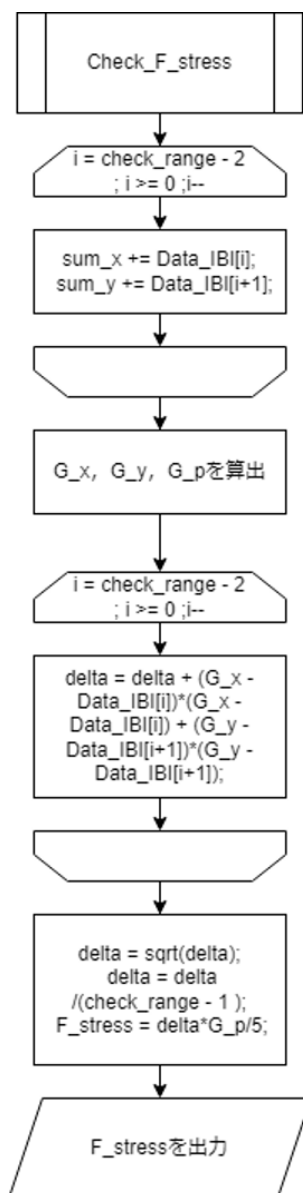


図7 Check_F_stress() 関数のフローチャート

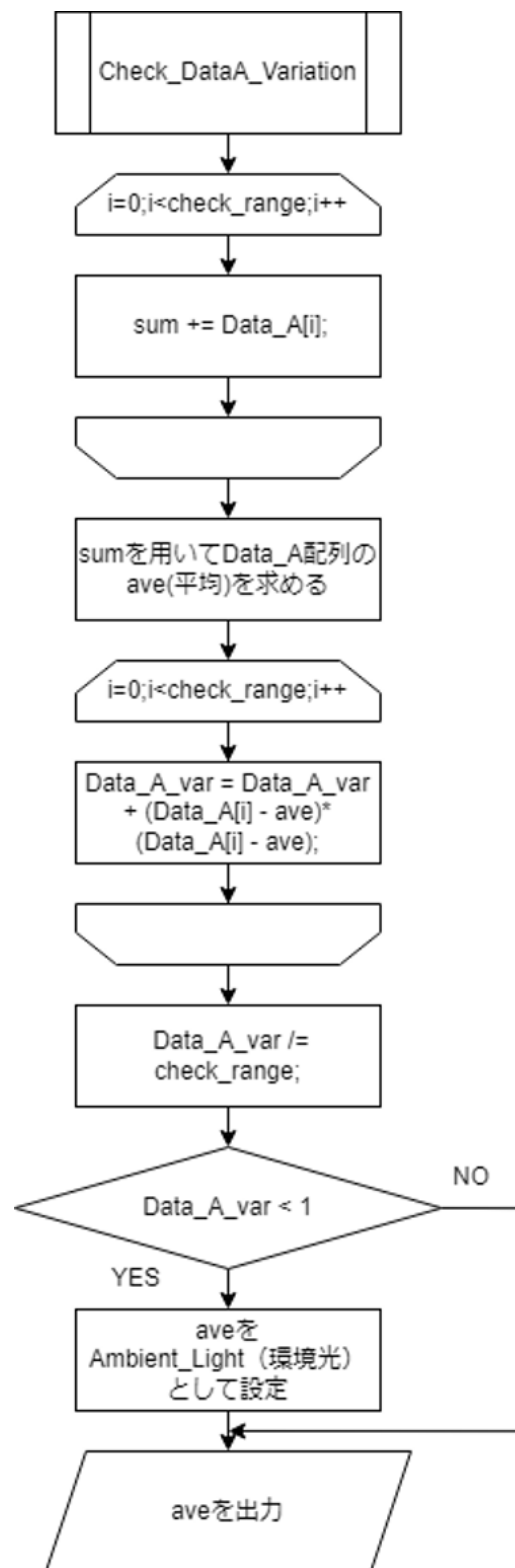


図 8 Check_DataA_Variation() 関数のフローチャート

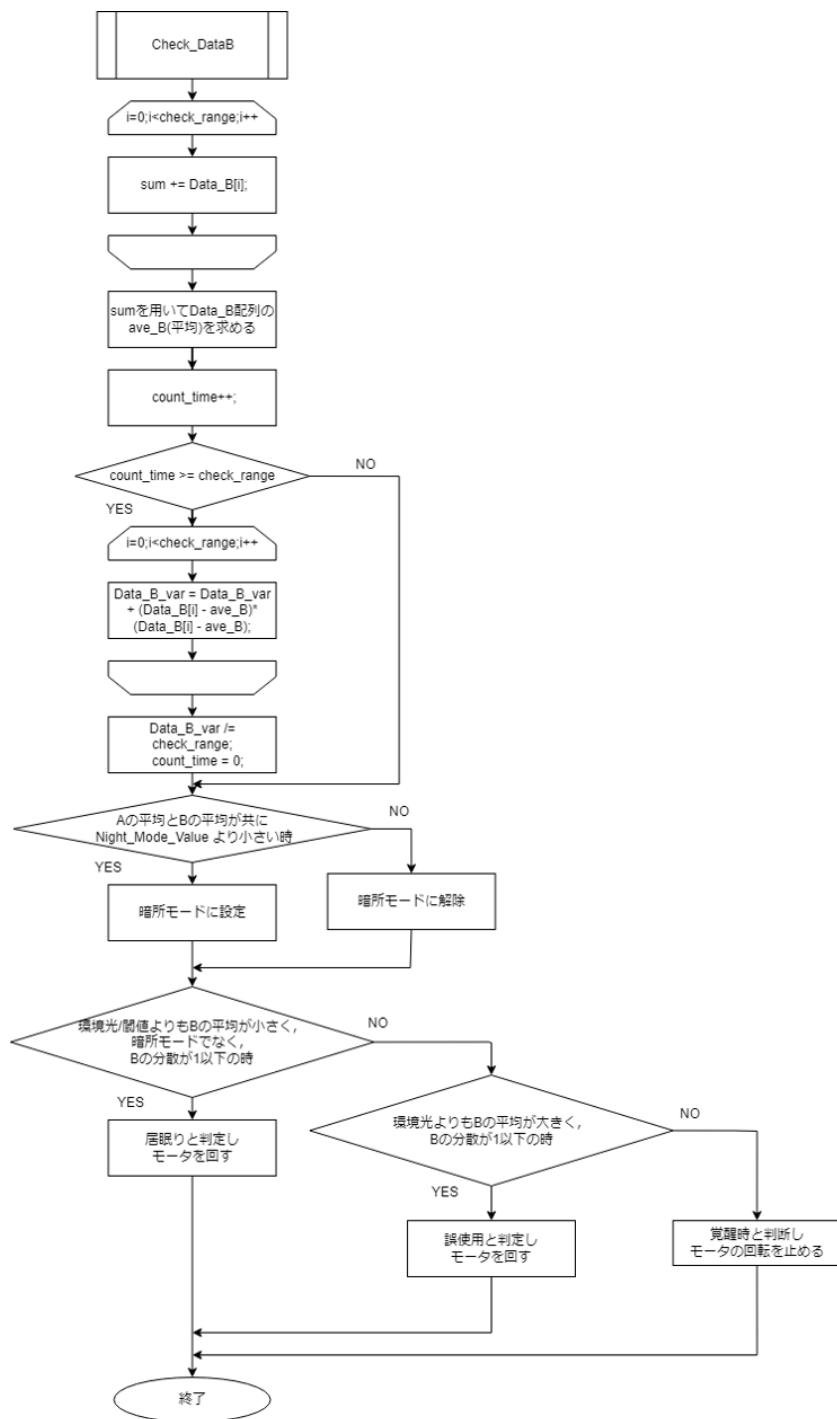


図 9 Check_DataB() 関数のフローチャート

```

1:   fill(255,0,0);
2:   textSize(16);
3:   if(Data_B_var != -1){
4:     Data_B_var_pr = Data_B_var;
5:     text("Data_B_var:" + str(Data_B_var),140,20);
6:   }else{
7:     text("Data_B_var:" + str(Data_B_var_pr),140,20);8:   }

```

図 10 ノイズ除去

表 3 主な関数と機能

機器	個数
Arduino Uno	1
ブレッドボード	4
DC モータ (FA-130RAL)	1
モータドライバ (TA7291P)	1
光センサ (Cds セル)	2
脈拍センサ (SFE-SEN-11574)	1
抵抗 (1k Ω)	2
抵抗 (10k Ω)	1

3 スケジュールと担当

本システムは、光センサによる居眠り検知と脈拍センサによる居眠り推定法部分の2つ部分の実装が必要になる。ここでは、光センサによる居眠り検知をタスク A、脈拍センサによる居眠り推定法をタスク B として、タスク A は、事前実験と検証 (A-1)、光量差による居眠り検知のプログラミング (A-2)、タスク B は、Check_F_stress() 関数の実装 (B-1)、Check_DataA_Variation() 関数の実装 (B-2)、Check_DataB() 関数の実装 (B-3)、loop() 関数の実装 (B-4)、回路製作と外装 (タスク C)、最終調整 (タスク D) として、それぞれ図 12 に示す担当で進めることとした。報告者はタス

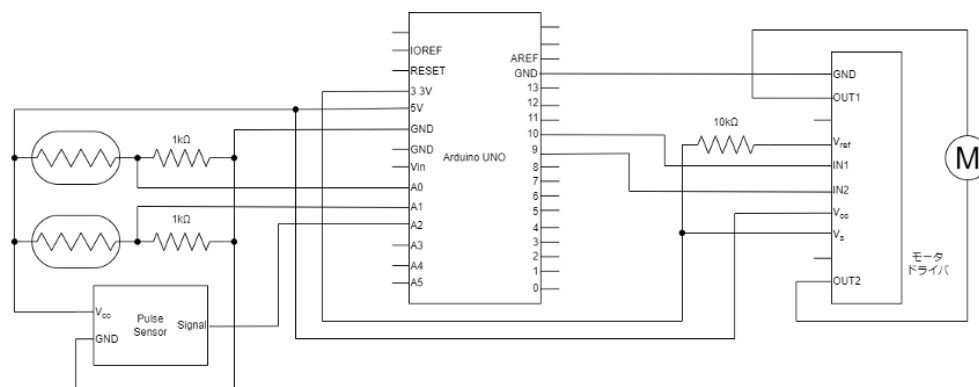


図 11 システムの回路図

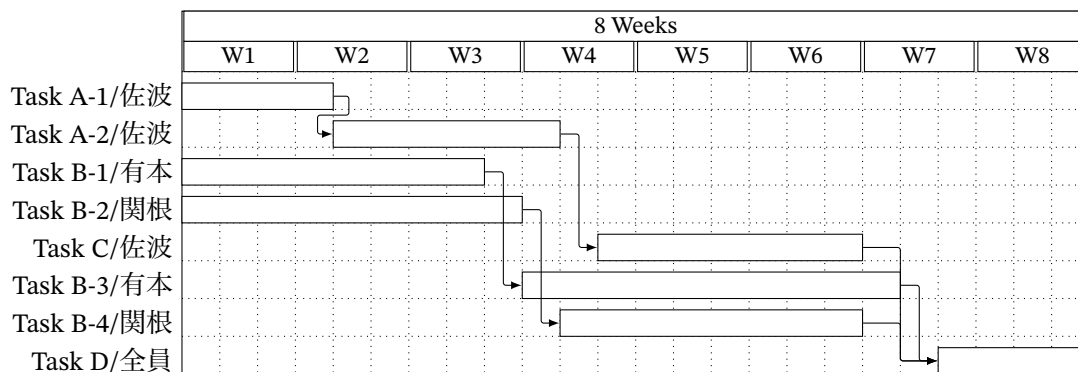


図 12 計画時のスケジュール

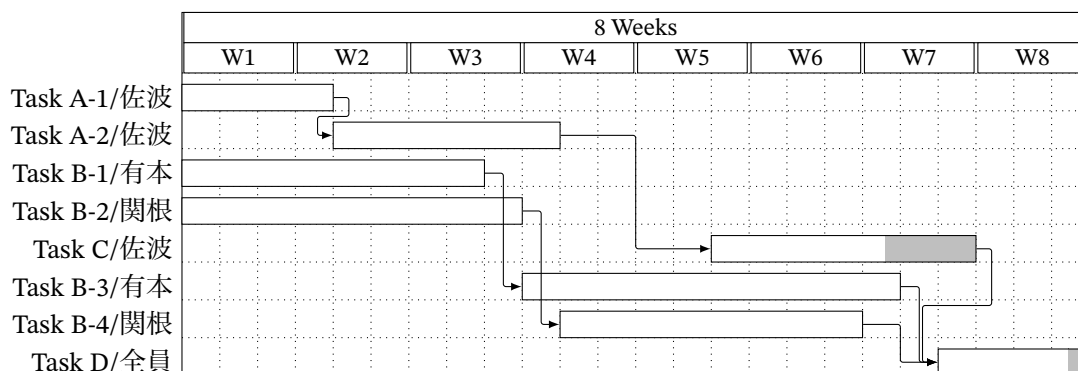


図 13 実際のスケジュール

ク A-1, A-2, C, D を担当した。

ただし、実際の作業では、タスク C における材料の調達に時間がかかり、製作の開始時期が遅れたことから、センサ機構の外装部分まで手が回らず剥き出しのままの実装となった。実際の進捗は、図 13 の通りとなった。プロジェクトのスケジュール立案時には、材料の調達期間を考慮する必要があることを身をもって知ることができたため、この経験は今後に活かしたい。

4 システムの動作確認

4.1 動作確認方法

他の学生に協力してもらい、動作確認を行った。具体的には、椅子に座ってもらい光センサ B を衣服に、脈拍センサを人差し指に装着した。Processing 側で表 4 に示す項目を表示し、条件ごとに必要な情報を記録した。動作確認をする条件は覚醒時、覚醒度低下時、起床時、誤使用時、暗所にいる時に分けて実施し、それぞれの確認事項、確認項目、確認方法を次の通りとした。

覚醒時

被験者が起きている時にセンサを取り付け記録する。

表 4 Processing で表示する項目と詳細

表示する項目	詳細
Mod	暗所モードなら 1, それ以外なら 0 を表示する
State	居眠り判定なら 1, 誤使用判定なら 2, それ以外なら 0 を表示する
Data_B_var	5 秒間ごとの光センサ B の分散を数値で表示する
Data_B	現在の光センサ B の値を数値で表示する
Ambient_Light	現在の環境光を数値で表示する
myBPM	現在の BPM を数値で表示する
LP	LP を表示する
RRI のグラフ	縦軸 RRI, 横軸時間のグラフを表示する
F-stress のグラフ	縦軸 F-stress, 横軸時間のグラフを表示する
光センサ A のグラフ	縦軸光センサの値, 横軸時間のグラフを表示する
光センサ B のグラフ	縦軸光センサの値, 横軸時間のグラフを表示する

• 確認事項

- RRI や F-stress がどのような値を取るのか
- LP の各点が理論通りに散らばって表示されるのか
- 光センサ A と B はどのように変化するか
- 具体的な環境光の値はいくつか

• 確認項目

- LP
- RRI
- F-stress のグラフ
- 光センサのグラフ
- Ambient_Light
- State

覚醒度低下時

被験者がうとうとしている時の確認項目を記録する。

• 確認事項

- RRI や F-stress がどのような値を取るのか
- LP の各点が理論通りに右上に集まって表示されるのか
- 光センサ A と B はどのように変化するか
- 具体的な環境光の値はいくつか

• 確認項目

- LP

- RRI
- F-stress のグラフ
- 光センサのグラフ
- Ambient_Light

起床時（居眠り判定）

被験者の覚醒度が低下していき、本装置によって居眠りが検知・防止された際の確認項目を記録する。

• 確認事項

- 本装置で居眠りを判定し、モータを回すことで実際に被験者を起こすことができたか
- 被験者が目覚めた後、モータは止まったか
- RRI や F-stress, 光センサのグラフには、起床前と起床後でどのような変化があるか
- LP は覚醒度低下時に比べ、どのように変化したか
- その時の具体的な環境光の値はいくつか
- 居眠り判定として State が 0 から 1 に変化したか
- 起床した際、State が 1 から 0 に戻ったか

• 確認項目

- LP
- RRI
- F-stress のグラフ
- 光センサのグラフ
- Ambient_Light
- モータの様子
- 被験者の様子
- State

誤使用時

わざと姿勢を悪くしてもらい、その時の確認項目を記録する。

• 確認事項

- 誤使用時（State = 2）を検知し、モータで警告できるか
- 被験者が使用法や姿勢を直した際、State は 0 に戻り、モータの回転は止まったか

• 確認項目

- 光センサのグラフ
- モータの様子
- State
- Data_B_var

暗所にいる時

本装置を稼働したまま、部屋の照明を消す。

- 確認事項
 - 暗所を検知し、Mode を 0 から 1 へ切り替えられるか
 - 暗所の際、モータが回転しないか
 - 光センサ A, B とともに値が低くなるか
- 確認項目
 - Mode
 - 光センサのグラフ
 - モータの様子

4.2 実行結果

4.2.1 覚醒時の動作

覚醒時の LP を図 14, RRI を図 15, F-stress を図 16, 光センサのグラフを図 17 に示す。この時の Ambient_Light の値は 54 であり、State は 0 であった。図 14 に示すように LP の各点は、200ms から 1000ms の間にあり、ばらつきが見られた。図 15 に示す RRI は 6 秒から 15 秒、39 秒から 50 秒にかけて変動があり、図 16 に示す F-stress も同様の時間に値の変動が見られた。図 17 に示す光センサ A の値は 0 秒から 50 秒にかけて 54 と変動はなかった。光センサ B のグラフは、頭の動きに合わせ、非周期的な波形をしていることが読み取れる。

4.2.2 覚醒度低下時の動作

覚醒度低下時の LP を図 18, RRI を図 19, F-stress を図 20, 光センサのグラフを図 21 に示す。この時の Ambient_Light の値は 55 であった。図 18 に示す LP の各点は、 $RRI_n - RRI_{n+1}$ 平面上において右上に集まってプロットされた。図 19 に示す RRI の値は 720ms から 900ms の間を動き、図 15 に示すほど変動していないことが読み取れる。図 20 に示す F-stress は、0 秒から 50 秒にかけて 0 に近い値を取っており、図 16 に比べると変動が小さい。図 21 に示す光センサ A の値は、0 秒から 50 秒にかけて 55 と変動しておらず、光センサ B のグラフは、図 17 と異なり、一定周期の波の形をとっていることが読み取れる。

4.2.3 起床時の動作

本装置によって起床した時の LP を図 22, RRI を図 23, F-stress を図 24, 光センサのグラフを図 25 に示す。この時、Ambient_Light の値は 49 であった。State は 28 秒に 0 から 1 へ、31 秒に 1 から 0 へと変化した。同時に 28 秒からモータが回転し、31 秒にモータの回転が止まった。図 22 に示す LP の各点の多くは、 $RRI_n - RRI_{n+1}$ 平面上において右上に集中していることが読み取れるが、図 18 に比べ、左下方向にばらつき始めていることがわかる。図 23 に示す RRI において、0 秒から

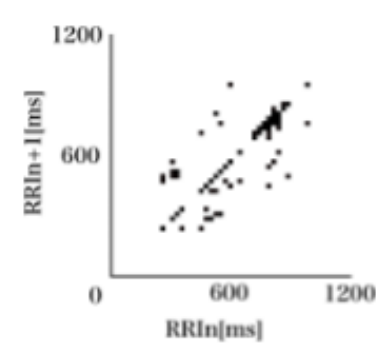


図 14 覚醒時の LP

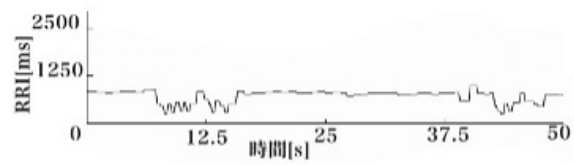


図 15 覚醒時の RRI

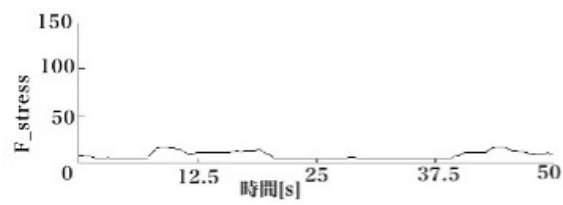


図 16 覚醒時の F-stress

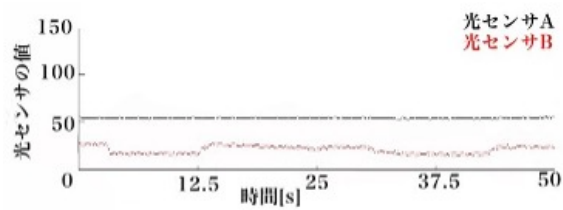


図 17 覚醒時の光センサ

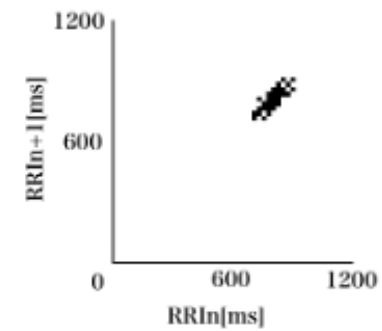


図 18 覚醒度低下時の LP

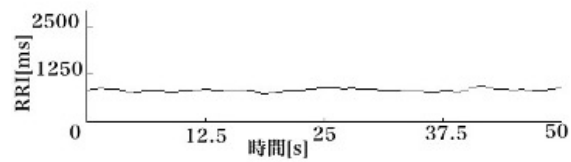


図 19 覚醒時低下時の RRI

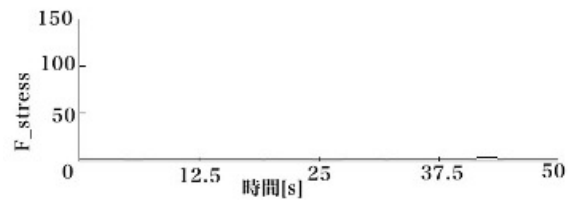


図 20 覚醒時低下時の F-stress

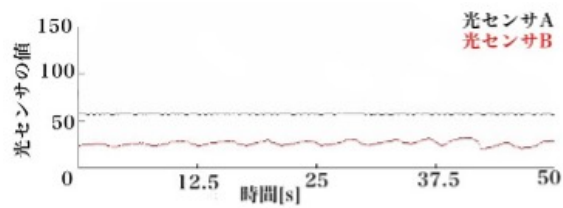


図 21 覚醒時低下時の光センサ

30.2 秒にかけて値が緩やかに変化しているが、30.2 秒から値の変動が大きくなっていることがわかる。同様に図 24 に示す F-stress の値は、0 秒から 30.5 秒にかけて 0 に近い値を取っていたが、30.5 秒から変動を見せ、値が 3 から 15 へと大きくなっていることがわかる。図 25 に示す光センサ A の値は 0 秒から 50 秒にかけて 49 と一定である。一方、光センサ B のグラフは 0 秒から 25 秒にかけて周期的な波形をしていた。しかし、25 秒から 27 秒にかけて値が徐々に小さくなった。27 秒から 29.5 秒にかけて 0 に近い値を取っていたが、29.5 秒から 37.5 秒かけて値が 0 から 20 へと増加していることが読み取れる。また、被験者は 0 秒から 25 秒にかけてうとうとしていたが、25 秒から 30 秒にかけて前かがみになり、30 秒から 31 秒の間に目覚めた。

4.2.4 誤使用時の動作

被験者は 30 秒から 31 秒にかけて光センサ B の値が光センサ A の値よりも大きくなるよう、姿勢を変えた。誤使用時の光センサのグラフを図 26 に示す。図 26 から光センサ B の値が 30 秒から 31 秒にかけて光センサ A の値よりも大きくなっていることがわかる。また、42 秒から 43 秒にかけて光センサ B の値が、光センサ A の値よりも小さくなっていることがわかる。

0 秒から 30 秒にかけて State は 0 であったが、31 秒から State は 2 に変わった。State が 2 に変わると同時にモータが回転を始めた。被験者は 42 秒から 43 秒にかけて姿勢を戻した。44 秒に State は 1 から 0 へと戻った。State が 0 に戻ると同時に、モータの回転が止まった。

4.2.5 暗所にいる際の動作

動作確認として 21 秒から 23 秒にかけて部屋の照明を消した。2 つの光センサのグラフを図 26 に示す。図 26 から光センサ A は 50、光センサ B は 24 の値を取っていたが、21 秒から 23 秒にかけて共に値が小さくなっていることが読み取れる。また、25 秒から 4 秒にかけて、両センサ共に 0 に近い値を取っていることがわかる。0 秒から 24 秒にかけて Mode は 0 であったが、24 秒から 25 秒にかけて Mode は 1 に変化した。Mode が 1 の時、モータは回らなかった。

次に 41 秒から 43 秒にかけて部屋の証明を点灯した。図 26 から 41 秒から 43 秒にかけて光センサ A と B の値が共に増加していることがわかる。41 秒から 43 秒の間に Mode は 1 から 0 へ変化した。

5 評価と考察

4.2 節で示した結果について評価及び考察を行う。

5.1 覚醒時と覚醒度低下時の RRI 変動の違いについて

4.2.2 節で述べたように図 19 に示す RRI の値は図 15 に比べ、安定している。逆に図 15 に示す RRI の値は図 19 に比べ、変動している。つまり、覚醒度低下時と覚醒時では、RRI 変動に違いが見られる。この理由について考える。呼吸周期が長くなると、RRI の変化が呼吸性変動に対して追従しやすくなると言われている [11]。覚醒度が低下すると、副交感神経が働くことにより、呼吸周期

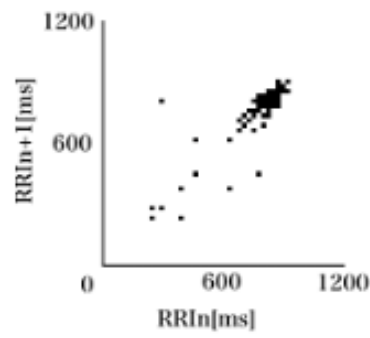


図 22 起床時の LP

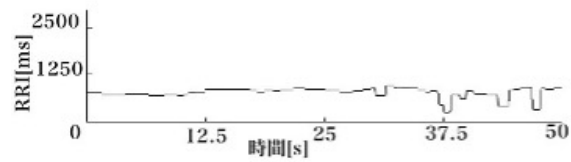


図 23 起床時の RRI

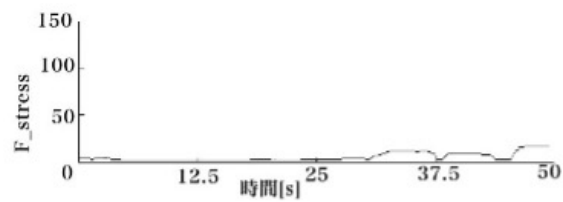


図 24 起床時の F-stress

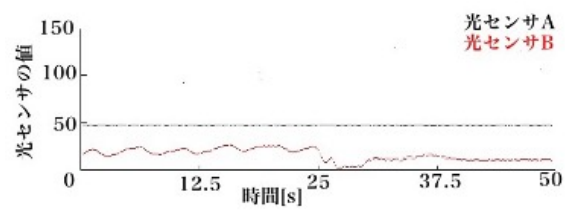


図 25 起床時の光センサ

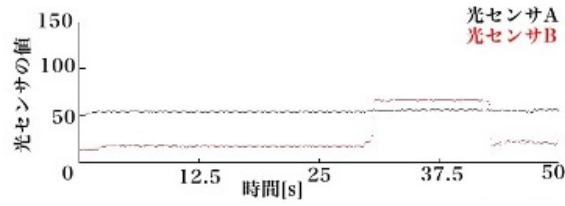


図 26 誤使用時の光センサ

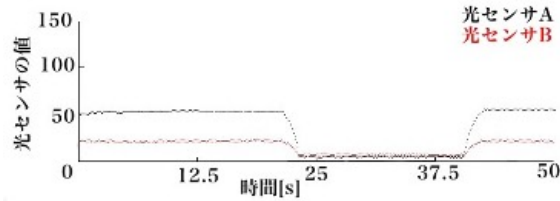


図 27 暗所にいる際の光センサ

は長くなり、呼吸パターンが安定する。よって、安定した呼吸パターンに RRI が追従するため、覚醒度低下時に RRI の値が安定すると考えられる。

一方、呼吸周期が短いと、RRI の変化が呼吸性変動に対して追従しにくいと言われている [11]。覚醒時には、交感神経が活発化し、ストレスや活動量などにより呼吸周期が短くなり、呼吸パターンが変動する。よって、RRI が安定したパターンに追従することがないため、覚醒時に RRI の値が変動すると考えられる。

5.2 本実験で用いた LP の信頼性について

2.2.2 節で述べたように、安静時、LP のプロットの重心が右上に推移し、緊張時は左下方向に推移しながら各点の散らばりが大きくなる。さらに緊張が高まると、プロットの重心は左下に推移しながら各点の散らばりが小さくなることが知られている [9]。

4.2.1 節で図 14 を用いて示したように、覚醒時の LP の各点には、200ms から 1000ms までのばらつきが見られた。また、4.2.2 節で図 18 を用いて示したように、覚醒度低下時の LP の各点は、 $RRI_n - RRI_{n+1}$ 平面上において右上に 720ms から 900ms の値の範囲に集まってプロットされた。これらは、2.2.2 節で述べた理論通りの結果となっている。よって、本実験で実装した LP は、被験者の覚醒度を幾何学的に正しく示していると言える。

5.3 F-stress の技術的課題について

2.2.3 節では、F-stress がゼロに近くなるほど、ストレスが高まっている [9] と述べた。しかし、覚醒度低下時にもかかわらず、4.2.2 節で述べた図 20 に示す F-stress の値は、0 秒から 50 秒にかけて 0 に近い値を取っている。この理由を考察する。

2.2.3 節で示した F-stress を求める計算式に注目する。(3) 式で示しているように F-stress は g_p と δ の積であった。ところで、 δ は各点から重心までの平均距離であり、重心と各点のばらつきを表す指標であった。覚醒度低下時は、5.1 節で述べたように RRI 変動は小さくなり、図 18 のように

LP 上の各点のばらつきは小さくなる。よって、覚醒度低下時の δ の値が 0 に近づくため、F-stress も図 20 のように 0 に近い値を取るのだと考えられる。

確かに、2.2.2 節で述べたように緊張度がかなり高いと $RRI_n - RRI_{n+1}$ 平面上において左下にプロットが集中することが知られている [9]。しかし、覚醒度低下時にも散らばりが小さくなるため、どちらの場合においても F-stress は 0 に近い値を取るようになる。よって、評価指標 F-stress は、ストレスが高い時と覚醒度低下時の区別ができないという技術的課題が、本実験の結果より明らかになった。

5.4 本装置の居眠り検知・防止の評価

起床時の結果を評価する。4.2.3 節で述べたように、図 22 に示す起床時の LP は、図 18 に示す覚醒度低下時の LP に比べ、各点がばらつき始めている。具体的には、720ms から 900ms の間にあったプロットが 200ms から 900ms の間に散らばり始めている。2.2.2 節で述べた理論から、本装置によって、安静時から被験者の緊張が徐々に高まっていると考えられる。また、4.2.3 節で述べたように、被験者はモータの振動によって 30 秒から 31 秒の間に目覚めている。実際に起床したという事実と 5.2 節で述べた信頼性のある幾何学的証拠から、本装置は居眠りの検知・防止に有効性があると言える。

5.5 F-stress に対する覚醒度解析における有効性

4.2.3 で述べたように、図 24 に示す F-stress は、0 秒から 30.5 秒においても 0 に近い値を取っていた。しかし、5.4 節で述べたように F-stress は覚醒度低下時とストレスが高い時を区別できない。一方、図 25 に示す光センサ B のグラフは 0 秒から 25 秒にかけて周期的な波形をしていた。4.2.2 節で示した図 21 の光センサ B のグラフも同様に周期的な波形をしていた。どちらも被験者がうとうとうとしていたことから、この周期的な波形は、頭のうとうとした動きを捉えていると考えられる。本システムは、このうとうとしている様子をグラフ上に捉えられるか否かにおいて、既存の覚醒度解析よりも有効性があると言える。

5.6 誤使用時、暗所にいる際の動作の評価

4.2.4 節の結果から、誤使用時の動作が 2.1.1 節で示した通りに動いていることがわかる。具体的には、光センサ B の値 > 光センサ A の値、光センサ A の値 > 光センサ B の値と切り替えることによって、正しく State が $0 \rightarrow 2$, $2 \rightarrow 0$ と切り替わっていることがわかる。また、モータの動作も同じように停止 $\rightarrow$ 回転、回転 $\rightarrow$ 停止と切替わっていることがわかる。よって、誤使用時のシステムは本装置に正しく実装できたと言える。

また同様に、4.2.5 節の結果から、暗所にいる際の動作が 2.1.1 節で示した通りに動いていることが分かる。具体的には、部屋の照明を点灯 $\rightarrow$ 消灯、消灯 $\rightarrow$ 点灯と切り替えることによって、正しく Mode が $0 \rightarrow 1$, $1 \rightarrow 0$ に切替わっていることがわかる。また、暗所において、モータが回らないことも確認できた。よって、暗所のシステムが本装置に正しく実装できたと言える。

6 おわりに

本実験では、昼食後のデスクワークにおける居眠りの防止を目的として、Arduino と光センサを用いた居眠り検知・防止装置を開発した。脈拍センサで取得した値を用いて LP と F-stress を表示し、被験者のストレス状態を幾何学的かつ数値的に扱った。それらを用いて本装置の居眠り検知・防止を検証したところ、本装置で居眠り検知・防止が可能であることが明らかになった。また、誤使用時や暗所時におけるシステムの動作の確認を行い、正しく実装できていることがわかった。システム側から誤使用を検知でき、既存の評価指標ではできなかったうとうとしている様子をグラフ化できる点において、本システムは有効性がある。しかし、うとうとしている時の頭の動きをグラフ上に表すことが出来たが、プログラム側でうとうと状態を判定するシステムは未実装である。うとうとしている状態をシステム側が検知できれば、深い居眠りに到達する前に被験者を起床させることができる可能性がある。その実装のためには、うとうとしている時の光センサの値とその時の LP のデータをより多く収集し、波形を解析する必要がある。

参考文献

- [1] 厚生労働省, “第 3 部 生活習慣調査の結果,” 令和元年国民健康・栄養調査報告, pp.186, 2019.
- [2] 大見拓寛, “運転者の居眠り状態評価の画像センサ,” 人工臓器 42 巻 1 号, pp.99-103, 2013.
- [3] 高木 和, 中山 治人, 岡田 志麻, 牧川 方昭, “頭部の動きに着目した運転時における覚醒度低下検出の可能性,” 生体医工学, pp.46-51, 2013.
- [4] 金子成彦, 藤田悦則, “ドライバーの覚醒低下警告・防止に向けた技術開発,” 特集長時間運転と疲労, pp.57-63, 2013.
- [5] Takumi MIYAZAWA, Ichiro FUKUMOTO, “生理指標を用いた居眠り防止システムの基礎検討,” Journal of Advanced Science, Vol.20, No.1&2, 2008.
- [6] 横山直樹, “居眠り検知を目的とした生理学的パラメータの研究,” ライフサポート, Vol.23, No.1, 2011.
- [7] 谷田 陽介, 萩原 啓, “心拍 RRI のローレンツプロット情報に着目した入眠移行気の簡易推定法,” 生体医工学, 156-162, 2006.
- [8] 豊福 史, 山口 和彦, 萩原 啓, “心電図 RR 間隔のローレンツプロットによる副交感神経活動の簡易推定法の開発,” 人間工学, Vol.43, No.4, 2007.
- [9] 松本 佳昭, 森 信彰, 三田尻 涼, 江 鐘偉, “心拍揺らぎによる精神的ストレス評価法に関する研究,” ライフサポート, Vol.22, No.3, 2010.
- [10] 日本自律神経学会, 自律神経機能検査 第三版, 文光堂, 2000.
- [11] 早野 純一郎, 岡田 暁宣, 安間 文彦, “心拍のゆらぎ: そのメカニズムと意義,” 人工臓器 25 巻 5 号, pp.870-880, 1996.

A Arduino のスケッチ

作成した Arduino のスケッチを以下に示す.

```
1:      // 光センサによる居眠り検知システム (Doze-detection system using optical
      sensors)
2:      #define USE_ARDUINO_INTERRUPTS true
3:      #include <PulseSensorPlayground.h>
4:      const int SENSOR_A = 0; //環境光(部屋の明るさ)
5:      const int SENSOR_B = 1; //頭の動きを検知
6:      const int SENSOR_Pulse = 2; //心拍を検知
7:      const int IN1 = 10; //モータ
8:      const int IN2 = 9; //モータ
9:      const int check_range = 50; //センサーの値をチェックする範囲, つまり配列の大き
      さ 5秒間
10:     const int A_dif = 2; //光センサAの値が一定になったと判別する際に用いる変数
11:     const int Night_Mode_Value = 15; //夜モードになるための閾値
12:     const float Judgment_Value = 4; //居眠りと判定するための変数 通常は環境光/3が
      頭の影 (自宅環境)
13:
14:     PulseSensorPlayground pulseSensor;
15:
16:     int Data_A[check_range] = {0};
17:     int Data_B[check_range] = {0};
18:     int Data_IBI[check_range] = {0}; //心拍間隔時系列
19:     int Data_Pulse;
20:     int Ambient_Light = 55; //居眠り判定の際に用いる環境光の値, Data_A[]の平均値
21:     int Night_Mode = 0; //夜モードがどうかのスイッチ 1で夜モード
22:     int State = 0; //居眠りになったら1
23:     int myBPM = 0;
24:     int Threshold = 550; // どの信号を「心拍」としてカウントするか、またどの信号
      を無視するかを決定する
25:     int count_time = 0; //カウントするために用いられる変数
26:     int Data_B_var = 120; //光センサBの分散
    ~~~以降, 本サンプルでは省略~~~
```